Activity Ranking API – Test Suite Walkthrough

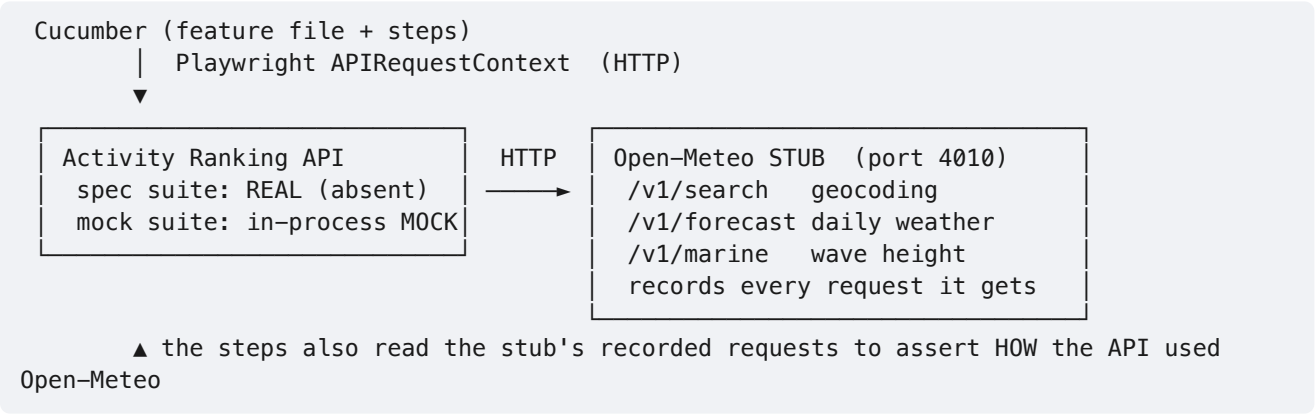
Spec-first BDD suite · Cucumber + Playwright API client + TypeScript · Open-Meteo handled with a local stub

1. The big picture

The ticket asks for a test suite written **before** the API exists. Two things had to be decided by us: the API contract and how to handle Open-Meteo. The repo contains two suites that share the same step code:

Suite	Command	Runs against	Result
Spec suite (21 scenarios)	`npm test`	The real API at `ACTIVITY_RANKING_API_URL` (default `http://127.0.0.1:3000`)	**Red** now (connection refused). Goes green once an implementation honours the contract. This is the intended spec-first state.
Mock suite (10 scenarios)	`npm run test:mock`	An in-process mock API + the Open-Meteo stub	**Green**. Proves the steps and assertions are valid and satisfiable.

2. How the pieces connect



- The tests start the **Open-Meteo stub** once (`BeforeAll`) and reset it before every scenario.
- A `Given Open-Meteo reports a snowy week for "Zermatt"` step configures the stub's weather for that city.
- The API under test must read `OPEN_METEO_BASE_URL` (set to `http://127.0.0.1:4010`) so its geocoding, forecast and marine calls hit the stub, not the internet.
- For `@mock` scenarios a hook starts the mock API on a free port and points the HTTP client at it; otherwise the client uses `ACTIVITY_RANKING_API_URL` .

3. File-by-file summary

Feature files (the specification)

File	Purpose
`features/activity-ranking.feature`	The main Gherkin spec: 21 scenarios (including outline rows) covering location search, ranking contract, weather behaviour, Open-Meteo usage and errors. Runs against the real API.
`features/mock/activity-ranking.mock.feature`	10 scenarios tagged `@mock`, a representative subset that runs against the mock API so there is a passing result.

Step definitions

File	Purpose
`features/step-definitions/activity-ranking.steps.ts`	All Given/When/Then code, shared by both suites. **Given**: configure stub weather or failures. **When**: call `/api/locations` or `/api/activity-rankings`. **Then**: assert status codes, response shape, ordering, weather-driven outcomes, reasoning text, and recorded Open-Meteo calls.

Support code

File	Purpose
`src/support/world.ts`	Cucumber World: owns the Playwright API client, last response and parsed body; exposes `get(path, params)`. Also defines TypeScript types for the contract (`ActivityResult`, `ForecastDay`, `LocationMatch`).
`src/support/hooks.ts`	Lifecycle: starts/stops the stub (`BeforeAll/AfterAll`), resets stub state and creates the HTTP client per scenario (`Before/After`), and starts the mock API for `@mock` scenarios.
`src/stubs/openMeteoStub.ts`	Local Open-Meteo stand-in. Built-in cities (Lisbon, Reykjavik, Oslo, San Francisco/Diego/Jose, Zurich), four weather profiles (snowy, sunny, rainy, surf), failure modes, request recording, and the seven expected dates. Response shapes mirror the real Open-Meteo.
`src/mock/mockActivityRankingApi.ts`	Small in-process implementation of the contract used only by the mock suite. Its scoring is deliberately simple and illustrative; it is not the system under test.

Configuration and docs

File	Purpose
`cucumber.js`	Two profiles: `default` (features/*.feature, real API) and `mock` (features/mock/*.feature).
`package.json`	Scripts: `test`, `test:mock`, `test:location`, `test:contract`, `test:weather`, `test:html`, `typecheck`.
`tsconfig.json`	Strict TypeScript settings for features and src.

`.env.example`	Documents `ACTIVITY_RANKING_API_URL`, `API_REQUEST_TIMEOUT_MS`, `OPEN_METEO_STUB_PORT`.
`README.md`	Approach, how to run, API contract, Open-Meteo strategy, assumptions and trade-offs.

4. Scenarios automated – spec suite (21)

Location search – `GET /api/locations?query=`

#	Scenario	What it proves
1	Suggest candidates for a partial name ("San")	200; exactly San Diego, San Francisco, San Jose; each has name, country, valid coordinates
2	Case and whitespace ignored (" sAn f ")	Query is trimmed and case-insensitive; returns only San Francisco
3	No candidates	200 with an empty list (not an error), good for autocomplete UX
4–6	Outline: query absent / empty / whitespace	400 with message saying query is required

Ranking contract – `GET /api/activity-rankings?location=`

#	Scenario	What it proves
7	Rank all four activities for seven days	Location echoed; dates equal the seven dates Open-Meteo served; every day has all four activities once; each result has date, rank, suitability, reasoning
8	Suitability range	Every score is a number 0–100
9	Ordering and ranks	Activities sorted by descending suitability; ranks run 1,2,3,4
10	Case/whitespace in city name (" IISBON ")	Still resolves to Lisbon

Weather behaviour (the core business rules)

#	Scenario	What it proves
11	Heavy snowfall	Skiing ranks first every day, beats Surfing; reasoning mentions "snow"
12	Clear mild weather (22°C)	Outdoor beats Indoor and Skiing; reasoning mentions "clear" and "22" (the ticket's example)
13	Heavy rain and strong wind	Indoor ranks first, beats Outdoor; outdoor reasoning mentions "rain"
14	Large waves, mild weather	Surfing beats Skiing and Indoor; reasoning mentions "wave"

Open-Meteo usage

#	Scenario	What it proves
15	Forecast fetched for resolved coordinates	The API geocoded "Lisbon" and requested a 7-day forecast at Lisbon's coordinates

Errors and validation

#	Scenario	What it proves
16–18	Outline: location absent / empty / whitespace	400 "location is required" and Open-Meteo is **not** called
19	Unknown city	404 "no location matched"; forecast not requested
20	Forecast service fails	502 "forecast data temporarily unavailable"
21	Geocoding service fails	502, same message

5. Scenarios automated – mock suite (10, all passing)

Same steps, run against the in-process mock: partial search; blank query rejected; full sunny-week ranking (shape, ordering, ranks, scores, "22" in reasoning); snow favours skiing; rain favours indoor; waves favour surfing; Open-Meteo called for geocoding and 7-day forecast; unknown city gives 404 with no forecast call; missing location gives 400 with no Open-Meteo call; forecast outage gives 502.

6. Key decisions and why

Decision	Reasoning
Test the API, never real Open-Meteo	Real forecasts change daily, which would make "snow favours skiing" flaky. A stub makes weather an explicit test input.
Stub instead of a test-only header	Failure injection happens at the dependency, so the product needs no test hooks that could leak to production.
API reads `OPEN_METEO_BASE_URL`	The one requirement placed on the future implementation so it can be redirected to the stub.
Two endpoints (`/api/locations` and `/api/activity-rankings`)	The ticket wants partial names to return possible matches. A separate autocomplete endpoint fits a front-end typeahead and keeps rankings unambiguous (exact city).
Empty match list is 200, unknown city ranking is 404	No suggestions while typing is normal; asking for a ranking of a non-existent place is an error.
Suitability is 0–100 plus an integer `rank`	The ticket says "a measure of suitability" without a scale; a bounded number is easy to display and test.
Relative assertions, not exact scores	The scoring formula is not specified; we pin intent (snow ⇒ skiing first) and leave the formula to the implementers.
Dates must equal what Open-Meteo returned	Ties the API output to its dependency; avoids hard-coding "today".
Reasoning keyword checks	Pins the ticket's examples ("High snowfall", "Clear skies and 22°C") without fixing exact wording.
Shared steps, separate mock feature	Gives a green run proving the assertions work, while the real-API spec stays red as the assignment expects.
Playwright API client with Cucumber	Lightweight HTTP client with good error output; no browsers needed.

7. Known limitations (documented in the README)

- Stub weather is the same for all seven days; mixed-week variation is a next step.
- Dates use the current UTC date; a run across midnight UTC could flake, and timezone policy is undecided.
- Slow/timeout provider behaviour, auth, rate limiting, caching and load are out of scope (not in the ticket).
- Equal scores and tie-breaking are not asserted.
- The contract is a proposal to confirm with the API owner.

8. How to run

```
npm install
npm run test:mock     # green: 10 scenarios, mock API
npm test              # red: 21 scenarios, real API not running (expected)

# when an implementation exists:
OPEN_METEO_BASE_URL=http://127.0.0.1:4010 npm start       # your API
ACTIVITY_RANKING_API_URL=http://127.0.0.1:3000 npm test
```